



Algorithm & C-Programming

Homework: The Clock Tower

For the engineering school: ECE-Paris



Infos

Edition:

1.0

 (2025.11.18)

Written by: Simon Céleste VANDAMME

svandamme@omnesintervenant.com



C. AI: RECOMMENDATIONS and DOWNSIDES

Artificial Intelligence (AI) tools show remarkable results, especially in the field of programming. Their integration into most Integrated Development Environments (IDEs) nowadays makes them more accessible than ever.

Used wisely, AI can be a valuable support in your journey to learn C programming. However, it can also be tempting to rely on it to complete entire projects or exercises in seconds.

When learning a new skill in any subject, remember this golden rule:

UNDERSTANDING > REWRITING PRE-DONE CONTENT

Note that I am NOT AGAINST the use of AI in your works.

However, I would highly appreciate it if you were transparent upon using it in your code and specify EVERY TIME why you are relying on it and how the snippet code works.

RECOMMENDATIONS

01. First think about the subject YOURSELF, write some elements down.
Then, use AI to continue your work if needed.

Your learning suffers when you let AI solve problems before trying yourself.
Skip the thinking, and you only train dependency... not skill.

As future engineers, you are expected to understand problems deeply and think creatively.
Keep training those skills: they are the foundation of your value as an engineer.

02. Use AI tools as personal teachers.
They are great to highlight your mistakes or improve your code quality.

Once done with a working program, you can prompt the AI on what is to be improved, for example:

- How to secure your code against errors?
- Ask for alternative ways to solve the problem. Did you use the most interesting way?
- How to improve your memory management or time execution?

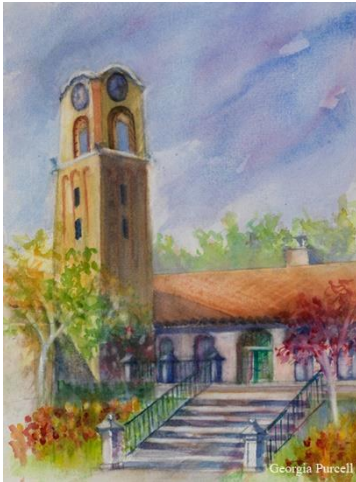
03. YOU are responsible for the code you submit... not the AI.

Whether at school or in your future job, your program is your creation. You must fully understand it and be able to explain every part of it, even if you received significant help from AI.

AI has limits: it struggles with complex projects, can produce insecure code, and lacks real creativity.
As an engineer, it is your job to go beyond these limits.



01. Context



The peaceful village of Bellecombe has lost its rhythm: the venerable clock on its tower has stopped.

Mr. Dubois, the mayor, a man deeply attached to traditions and particularly demanding, has entrusted you with a crucial mission: design a digital time-display system to replace the failing mechanism.

He insists on a reliable clock, simple to use, and strictly compliant with his precise specifications, because in Bellecombe, punctuality is a serious matter.

Your first step is to develop an initial version of this digital clock, as detailed in the requirements document he has given you.

02. First Version of the Clock

Your program must follow the rules below:

1. **At initialization**, the user must enter the time into the system.

The input is made in **ONE single command**, in either one of the following two formats:

```
Valeurs d'entree :  
13:04  
08h36
```

2. You must ensure that the values provided are correct:
 - hours < 24
 - minutes < 60
3. Create a loop in which you display the current time.
The program **remains fully stopped UNTIL** the user presses a key.
4. When the user enters anything in the console, advance the time by **+1 minute**.
5. The clock stays inside the loop until the user enters the following command:
"adm_exit" (for *admin_exit*).



03. Festive Version of the Clock

Now that your clock works, Mr. Dubois informs you that certain times in the village are very important and must absolutely be displayed when they occur.

1. Mealtimes

The clock must indicate when it is mealtime:

- **Breakfast:** from 7:30 to 8:00
- **Lunch:** between 11:30 and 13:00
- **Dinner:** from 18:30 to 20:00

2.1. Perfect Mirror Times

The clock must indicate the following “perfect mirror times”:

- 11:11
- 22:22
- 00:00

2.2. Imperfect Mirror Times

These correspond to any moment where there is a **symmetry or parallel pattern** between the hour and the minutes.

Examples (non-exhaustive):

- 12:21
- 03:30
- 15:50
- ...



04. The Clock Bell 🎵

Always attentive to the well-being of his residents, and impressed by your success with the time display, Mr. Dubois gives you a new task.

He wants the clock to emit a **brief sound signal** at key moments during the day.

The old tower clock used to chime "**N**" **times** for the hour "**NN h**" when a new hour began.

In other words:

- At **13:00**, the clock must chime 13 times.
- At **midnight (00:00)**, the clock does **not** chime.

This only happens on **full hours**, i.e., when the minutes are **00**.
(Examples: 11:00, 18:00...)

>> To simulate sound, you may simply display a special marker in the console at the appropriate time (for example, printing "**\ding**" repeatedly).

Thus, for **04:00**, you would output:

`"\ding \ding \ding \ding"`

05. A Clock That Keeps Time Properly

Rather than having someone press a key every minute, Mr. Dubois asks for a **final version** in which the user only needs to initialize the clock, and it will then advance **automatically**, without requiring any console input.

1. Create **two versions** of your clock:

- A manual one (your current version)
- An automatic one, which increments the time on its own

2. Select which version of the clock you want to launch, with a menu at the beginning of your program.

Additional help:

If you get stuck inside the automatic clock loop, you may force-stop the execution in a terminal using **CTRL + C**.